



UNIVERSIDAD POLITÉCNICA DE PACHUCA

Ing. en Tecnologías De La Información Y La Innovación Digital

REPORTE DE PRÁCTICA

Filas en Jetpack Compose: maquetación horizontal con Row, Arrangement, Modifier y LazyRow en Android Studio

Asignatura: Desarrollo de aplicaciones móviles

Alumno: Lorenzana Hernández Aarón Dahir

Matrícula: 2531126341

Grupo: TIID 4-02

Profesor(a): MIGUEL ANGEL MONTOYA CERRO

Pachuca de Soto, Hidalgo, México, 02/10/2026

Índice de contenido

1. Introducción	1
1.1 Contextualización	1
1.2 Planteamiento del problema u objeto de estudio	1
1.3 Objetivos	2
1.4 Justificación	2
1.5 Estructura del documento	3
2. Marco teórico	4
2.1 Definición de conceptos clave	4
2.2 Teorías y modelos relevantes	5
2.3 Antecedentes	6
2.4 Fuentes de información	6
3. Desarrollo	7
3.1 Preparación del código base con LazyColumn	7
3.2 Implementación del contenedor horizontal Row	9
3.3 Alineación y distribución con horizontalArrangement y Modifier	11
3.4 Aplicación de márgenes internos (padding) y saturación de elementos	13
3.5 Implementación de desplazamiento horizontal con LazyRow	15
3.6 Modularización en funciones Composable y prueba de scroll bidireccional	17
3.7 Análisis de resultados	19
4. Conclusiones	20
4.1 Aportes	20
4.2 Limitaciones	20
4.3 Recomendaciones y proyecciones	20
5. Referencias	21

6. Anexos	22
Anexo A. Datos del video consultado	22

Índice de ilustraciones

Ilustración 1: Proyecto practica2 abierto en Android Studio retomando el LazyColumn previo	8
Ilustración 2: Estructura base en MainActivity.kt con imagen y textos personalizados	9
Ilustración 3: Vista previa inicial del contenedor vertical en el emulador	9
Ilustración 4: Declaración del contenedor Row con tres componentes Text en el código	10
Ilustración 5: Visualización inicial de los textos apilados horizontalmente sin separación	11
Ilustración 6: Comprobación del tamaño envolvente (wrap content) al usar Arrangement.Center	12
Ilustración 7: Expansión de la fila a todo el ancho con Modifier.fillMaxWidth()	13
Ilustración 8: Distribución equitativa de elementos con Arrangement.SpaceBetween	14
Ilustración 9: Aplicación de Modifier.padding(16.dp) y formateo de código	14
Ilustración 10: Adición de múltiples etiquetas Text dentro del contenedor Row	15
Ilustración 11: Problema de saturación y corte visual en la fila estática sin scroll	16
Ilustración 12: Sustitución de Row por LazyRow e incorporación del bloque item	17
Ilustración 13: Modularización del código mediante la función MyRowExample()	18
Ilustración 14: Prueba del desplazamiento horizontal (scroll) en el emulador	18
Ilustración 15: Funcionamiento simultáneo del scroll vertical y horizontal al rotar la pantalla	19

Índice de tablas

Tabla 1: Comparación de maquetación horizontal entre vistas XML y Jetpack Compose	5
Tabla 2: Herramientas y tecnologías utilizadas	7
Tabla 3: Modificadores y propiedades aplicados a las filas de la práctica	15
Tabla 4: Relación entre objetivos específicos y evidencias	19

1. Introducción

1.1 Contextualización

En el diseño de interfaces declarativas para Android con Jetpack Compose, estructurar pantallas únicamente mediante columnas verticales resulta insuficiente para construir aplicaciones modernas. La mayoría de las vistas comerciales requieren combinar el apilamiento vertical con la disposición horizontal de elementos, tales como barras de categorías, etiquetas informativas, botoneras o carruseles deslizables.

Mientras que en el enfoque clásico basado en XML la distribución horizontal se implementaba mediante un `LinearLayout` con orientación horizontal o un `RecyclerView` configurado con un `LinearLayoutManager` horizontal, Jetpack Compose resuelve esta necesidad mediante los componentes `Row` y `LazyRow`. Dominar la maquetación por filas y su integración dentro de columnas existentes permite crear diseños bidireccionales adaptables a cualquier pantalla.

1.2 Planteamiento del problema u objeto de estudio

El objeto de estudio de esta práctica es el tercer episodio del curso de Jetpack Compose del canal Programación by AristiDevs (Guimerá Orozco, 2021), titulado *FILAS (ROW) en JETPACK COMPOSE*. La práctica consiste en retomar el proyecto de la sesión anterior (compuesto por un `LazyColumn` con una imagen de Solaire de Astora y textos descriptivos) e integrar en su parte inferior una fila horizontal de textos.

A lo largo de la práctica se analiza por qué los elementos dentro de un `Row` aparecen inicialmente pegados entre sí debido al ajuste envolvente por defecto, cómo distribuirlos proporcionalmente en todo el ancho mediante `Modifier.fillMaxWidth()` y `Arrangement.SpaceBetween`, cómo agregar márgenes internos con `padding(16.dp)` y, finalmente, cómo solucionar el desbordamiento cuando existen demasiados elementos en la fila reemplazando `Row` por un `LazyRow`.

1.3 Objetivos

Objetivo general. Implementar y configurar contenedores horizontales `Row` y `LazyRow` dentro de una interfaz vertical en Android Studio utilizando Jetpack Compose, siguiendo el tercer video del curso de AristiDevs, para organizar elementos en el eje horizontal con espaciado uniforme y soporte de desplazamiento.

Objetivos específicos.

1. Explicar el funcionamiento de los contenedores `Row` y `LazyRow` en Jetpack Compose y su equivalencia frente a los componentes horizontales del sistema XML.
2. Integrar un contenedor `Row` al final del `LazyColumn` desarrollado en la práctica previa para disponer múltiples componentes `Text` en una misma línea horizontal.

3. Configurar la alineación y distribución en el eje horizontal mediante la propiedad `horizontalArrangement` en conjunto con el modificador `Modifier.fillMaxWidth()`.
4. Aplicar márgenes internos mediante `Modifier.padding(16.dp)` para evitar que los textos queden pegados a los bordes físicos de la pantalla.
5. Comprobar el problema de saturación visual al insertar múltiples elementos en un `Row` estático y solucionarlo migrando a un contenedor `LazyRow` con bloques `item`.
6. Modularizar la fila en una función `@Composable` independiente (`MyRowExample`) y verificar el desplazamiento tanto vertical como horizontal en el emulador.

1.4 Justificación

En el ámbito académico, esta práctica complementa el aprendizaje de la maquetación básica en la asignatura de Desarrollo de aplicaciones móviles, integrando el eje horizontal con el eje vertical estudiado previamente para construir interfaces completas.

En el ámbito profesional, la combinación de una lista vertical (`LazyColumn`) que contiene listas horizontales (`LazyRow`) constituye el patrón de arquitectura visual más utilizado en aplicaciones móviles de streaming, comercio electrónico y redes sociales.

En el ámbito personal, comprender las reglas de alcance de las funciones `@Composable` y el manejo de estados de desplazamiento evita errores de compilación y permite estructurar código limpio y modular en Kotlin.

1.5 Estructura del documento

Este documento consta de seis secciones. La primera aborda la introducción, el planteamiento del problema, los objetivos y la justificación. La segunda desarrolla el marco teórico sobre filas, arreglos horizontales y listas perezosas. La tercera documenta el desarrollo práctico paso a paso con fragmentos de código, tablas y 15 ilustraciones de evidencia. La cuarta presenta las conclusiones, la quinta enlista las referencias en formato APA y la sexta incluye los anexos.

2. Marco teórico

En este apartado se describen los conceptos fundamentales requeridos para maquetar componentes sobre el eje horizontal en Jetpack Compose.

2.1 Definición de conceptos clave

Row (Fila). Contenedor de diseño en Jetpack Compose que organiza a sus elementos hijos de forma secuencial en el eje horizontal, uno al lado del otro de izquierda a derecha. Equivale a un `LinearLayout` con orientación horizontal (Android Developers, s. f.-a).

horizontalArrangement (Disposición horizontal). Parámetro de los contenedores horizontales (`Row` y `LazyRow`) que define cómo se distribuyen los elementos hijos a lo largo del eje principal cuando existe espacio libre disponible. Entre sus opciones destacan `Arrangement.Center`, `Arrangement.SpaceBetween`, `Arrangement.SpaceEvenly` y `Arrangement.spacedBy()` (Android Developers, s. f.-a).

Modifier.fillMaxWidth(). Modificador que instruye al componente para que ocupe el 100% del ancho permitido por su contenedor padre, indispensable para que las propiedades de distribución horizontal surtan efecto visual (Android Developers, s. f.-c).

Modifier.padding(). Modificador que inserta espacio en blanco (margen) alrededor o en el interior de un componente utilizando unidades independientes de densidad como `16.dp`, mejorando la separación respecto a los bordes de la pantalla (Android Developers, s. f.-c).

LazyRow e item. `LazyRow` es una fila con desplazamiento horizontal integrado que compone y renderiza únicamente los elementos visibles en pantalla. Al igual que su contraparte vertical, requiere encapsular los componentes dentro del DSL de listas mediante bloques `item { ... }` o `items(...)` (Android Developers, s. f.-d).

2.2 Teorías y modelos relevantes

Distribución espacial en el eje horizontal y restricciones de scroll. Por defecto, un contenedor `Row` envuelve su tamaño al ancho exacto que suman sus hijos (*wrap content*). Por ello, si se configura un arreglo como `Arrangement.Center` o `Arrangement.SpaceBetween` sin expandir previamente el ancho de la fila con `Modifier.fillMaxWidth()`, no se percibe ningún cambio en pantalla porque no hay espacio sobrante que repartir.

Por otra parte, cuando se anidan contenedores desplazables en Jetpack Compose, no se debe aplicar el modificador `.verticalScroll(rememberScrollState())` directamente a un `LazyColumn`, ya que los componentes perezosos poseen su propio motor de scroll interno y combinarlos genera una excepción de restricciones infinitas. En cambio, un `LazyColumn` vertical puede alojar en su interior un `LazyRow` horizontal sin conflicto alguno, puesto que sus ejes de desplazamiento son perpendiculares. La Tabla 1 compara la maquetación horizontal entre XML y Jetpack Compose.

Tabla 1: Comparación de maquetación horizontal entre vistas XML y Jetpack Compose

Aspecto	Sistema de vistas XML	Jetpack Compose
Contenedor horizontal estático	<code><LinearLayout android:orientation="horizontal"></code>	<code>Row { ... }</code>
Lista horizontal con scroll y reciclaje	<code>RecyclerView + LinearLayoutManager(HORIZONTAL)</code>	<code>LazyRow { item { ... } }</code>
Distribución proporcional del espacio	<code>android:layout_weight="1"</code> en cada vista hija	<code>horizontalArrangement = Arrangement.SpaceBetween</code>
Ocupar todo el ancho	<code>android:layout_width="match_parent"</code>	<code>Modifier.fillMaxWidth()</code>
Margen respecto a los bordes	<code>android:padding="16dp"</code>	<code>Modifier.padding(16.dp)</code>

2.3 Antecedentes

En el sistema tradicional de Android, construir una pantalla con desplazamiento vertical que incluyera además un carrusel horizontal en su interior implicaba implementar un `RecyclerView` anidado dentro de otro `RecyclerView` o un `NestedScrollView`, requiriendo múltiples adaptadores y archivos XML. Con Jetpack Compose, esta estructura se resuelve combinando `LazyColumn` y `LazyRow` en unas pocas líneas de código declarativo en Kotlin.

En la secuencia del curso de AristiDevs (Guimerá Orozco, 2021), después de haber creado una columna desplazable en el capítulo anterior, el tercer video introduce las filas para superar la limitación de dibujar únicamente hacia abajo.

2.4 Fuentes de información

El marco conceptual de este reporte se apoya en la documentación técnica oficial de Android Developers sobre diseños básicos, modificadores y listas perezosas, así como en el tercer video del curso de AristiDevs empleado como guía de laboratorio.

3. Desarrollo

A continuación se documenta paso a paso el desarrollo de la práctica integrando las filas en el código del proyecto `com.example.practica2`. La Tabla 2 resume las herramientas empleadas.

Tabla 2: Herramientas y tecnologías utilizadas

Herramienta	Versión / Detalle	Uso en la práctica
Android Studio	IDE Oficial Android	Edición del archivo <code>MainActivity.kt</code> , vista previa y compilación.
Kotlin	Lenguaje base	Declaración de funciones <code>@Composable</code> y paso de parámetros.
Jetpack Compose	UI Toolkit Declarativo	Implementación de <code>LazyColumn</code> , <code>Row</code> , <code>LazyRow</code> , <code>Arrangement</code> y <code>Modifier</code> .
Material Design 3	<code>androidx.compose.material3</code>	Renderizado de etiquetas <code>Text</code> y aplicación de colores (<code>Color.White</code> , <code>Color.Black</code>).
Emulador de Android (AVD)	Dispositivo Virtual	Comprobación del scroll horizontal y vertical en la aplicación.

3.1 Preparación del código base con LazyColumn

El desarrollo partió del proyecto `practica2` realizado en la sesión anterior, el cual cuenta con un contenedor `LazyColumn` de fondo negro (`Color.Black`) que aloja la imagen `R.drawable.images` ("Solair de astora") y cinco textos personalizados en color blanco (Ilustraciones 1, 2 y 3, y Bloque de código 1).

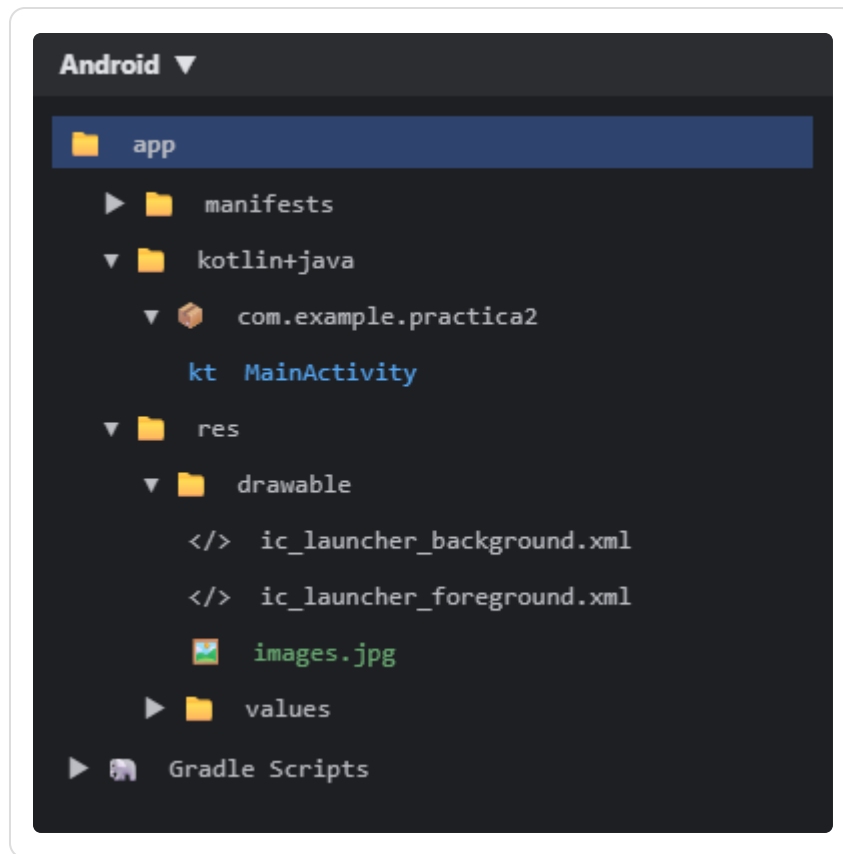


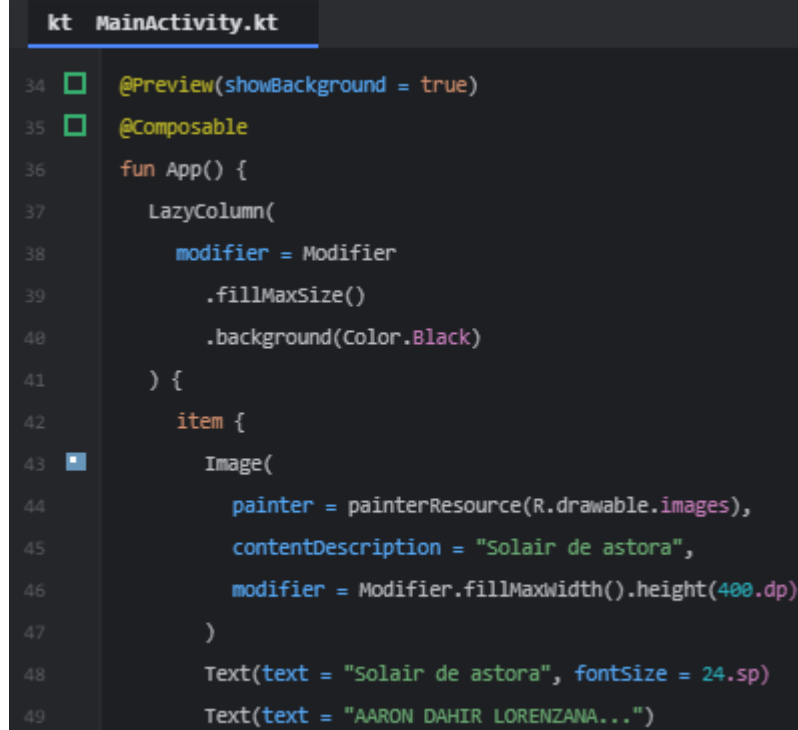
Ilustración 1: Proyecto practica2 abierto en Android Studio retomando el LazyColumn previo

Bloque de código 1. Código base del proyecto antes de insertar la fila

```
1 @Preview(showBackground = true)
2 @Composable
3 fun App() {
4     LazyColumn(
5         modifier = Modifier
6             .fillMaxSize()
7             .background(Color.Black)
8     ) {
9         item {
10             Image(
11                 painter = painterResource(id = R.drawable.images),
12                 contentDescription = "Solair de astora",
13                 modifier = Modifier
14                     .fillMaxWidth()
15                     .height(400.dp)
16             )
17             Text(text = "Solair de astora", fontSize = 24.sp, color = Color.White)
18             Text(
19                 text = "AARON DAHIR LORENZANA HERNANDEZ",
20                 fontSize = 32.sp,
21                 color = Color.White,
22                 modifier = Modifier.fillMaxWidth(),
23                 textAlign = TextAlign.Center
```

```
24         )
25         Text(text = "HOLA MUNDO000000", color = Color.White)
26         Text(text = "PRACTICA 2", color = Color.White)
27         Text(text = "Me gustan los pixel musho", color = Color.White)
28     }
29 }
30 }
```

Descripción: Se mantiene el `LazyColumn` sin el modificador `verticalScroll` para evitar conflictos con el motor de desplazamiento interno de la lista perezosa.



```
kt MainActivity.kt
34 @Preview(showBackground = true)
35 @Composable
36 fun App() {
37     LazyColumn(
38         modifier = Modifier
39             .fillMaxSize()
40             .background(Color.Black)
41     ) {
42         item {
43             Image(
44                 painter = painterResource(R.drawable.images),
45                 contentDescription = "Solair de astora",
46                 modifier = Modifier.fillMaxWidth().height(400.dp)
47             )
48             Text(text = "Solair de astora", fontSize = 24.sp)
49             Text(text = "AARON DAHIR LORENZANA...")
50         }
51     }
52 }
```

Ilustración 2: Estructura base en MainActivity.kt con imagen y textos personalizados

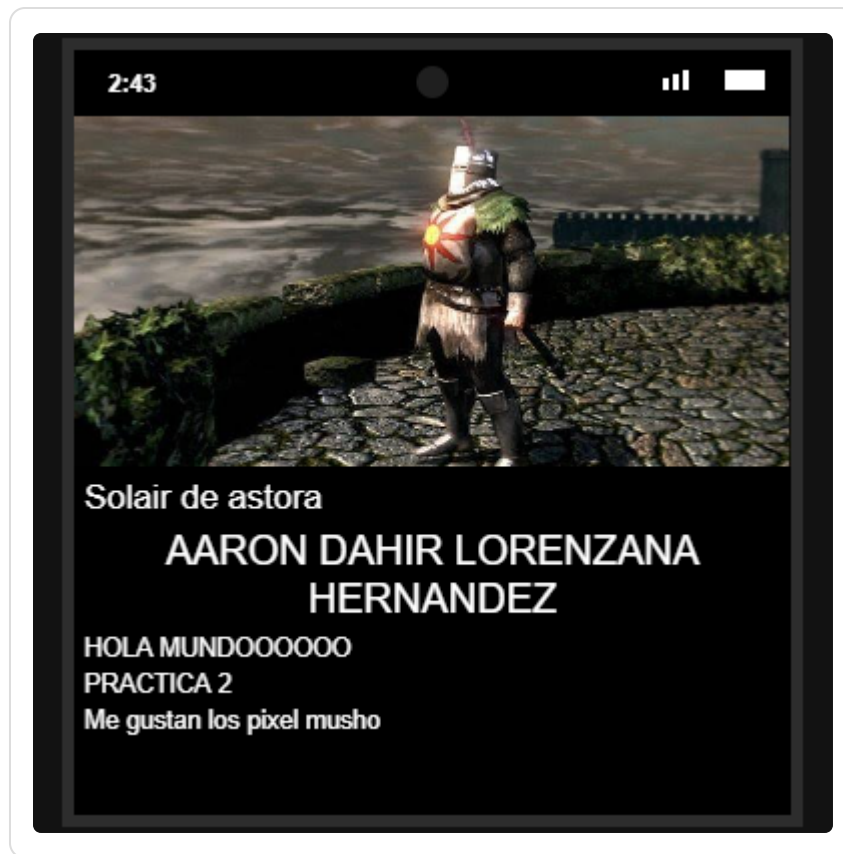


Ilustración 3: Vista previa inicial del contenedor vertical en el emulador

3.2 Implementación del contenedor horizontal Row

Siguiendo la explicación del video tutorial, al final de los textos del bloque `item` se declaró un componente `Row`. Dentro de sus llaves se agregaron tres componentes `Text` en color blanco con las cadenas "Stack", "Java" y "KOTLIN" (Bloque de código 2 e Ilustración 4).

Bloque de código 2. Creación del contenedor Row básico con tres textos

```
1 Row {  
2     Text(text = "Stack", color = Color.White)  
3     Text(text = "Java", color = Color.White)  
4     Text(text = "KOTLIN", color = Color.White)  
5 }
```

Descripción: El contenedor `Row` coloca los tres textos uno al lado del otro sobre la misma línea horizontal, evitando que se apilen hacia abajo o se superpongan.



Ilustración 4: Declaración del contenedor Row con tres componentes Text en el código

Al refrescar la vista previa, se observó que los tres textos se dibujaron en fila horizontal en la esquina inferior izquierda, pero completamente pegados entre sí (`StackJavaKOTLIN`), dificultando su lectura (Ilustración 5).

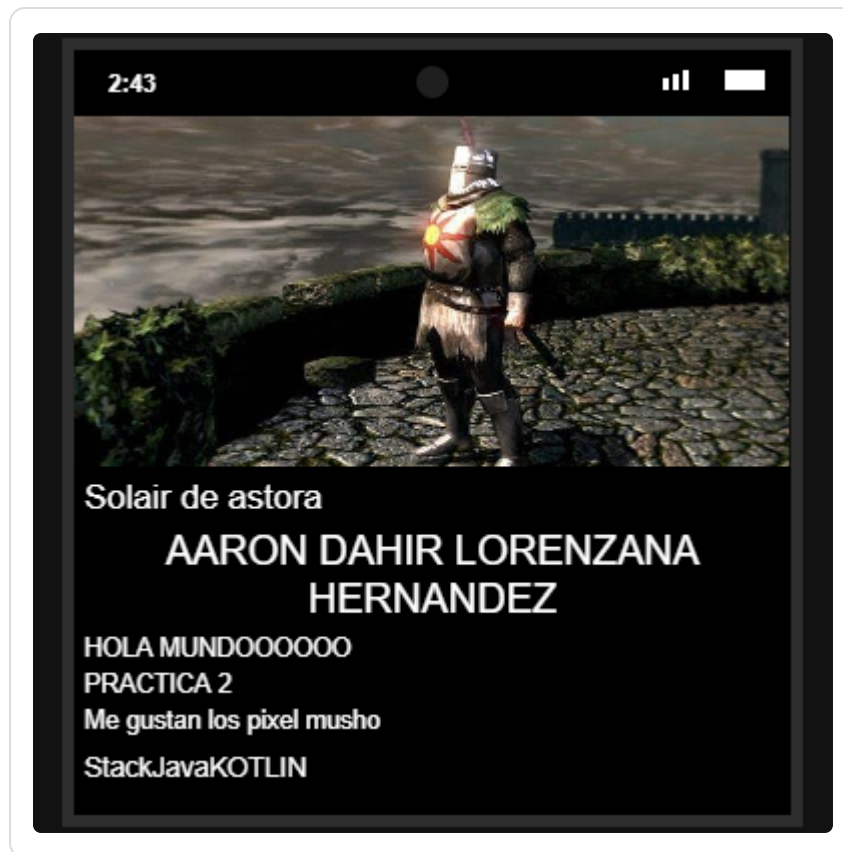


Ilustración 5: Visualización inicial de los textos apilados horizontalmente sin separación

3.3 Alineación y distribución con `horizontalArrangement` y `Modifier`

Para intentar centrar y separar los textos de la fila, primero se agregó el parámetro `horizontalArrangement = Arrangement.Center` al contenedor `Row` (Bloque de código 3). Sin embargo, al compilar la vista previa no ocurrió ningún cambio visual, ya que el `Row` mide por defecto únicamente el ancho de las tres palabras (*wrap content*) y no dispone de espacio libre hacia los lados (Ilustración 6).

Bloque de código 3. Prueba de `Arrangement.Center` sin modificador de ancho

```
1 Row(horizontalArrangement = Arrangement.Center) {  
2     Text(text = "Stack", color = Color.White)  
3     Text(text = "Java", color = Color.White)  
4     Text(text = "KOTLIN", color = Color.White)  
5 }
```

Descripción: Sin un modificador que expanda el ancho de la fila, `Arrangement.Center` no tiene espacio adicional para desplazar los textos al centro de la pantalla.

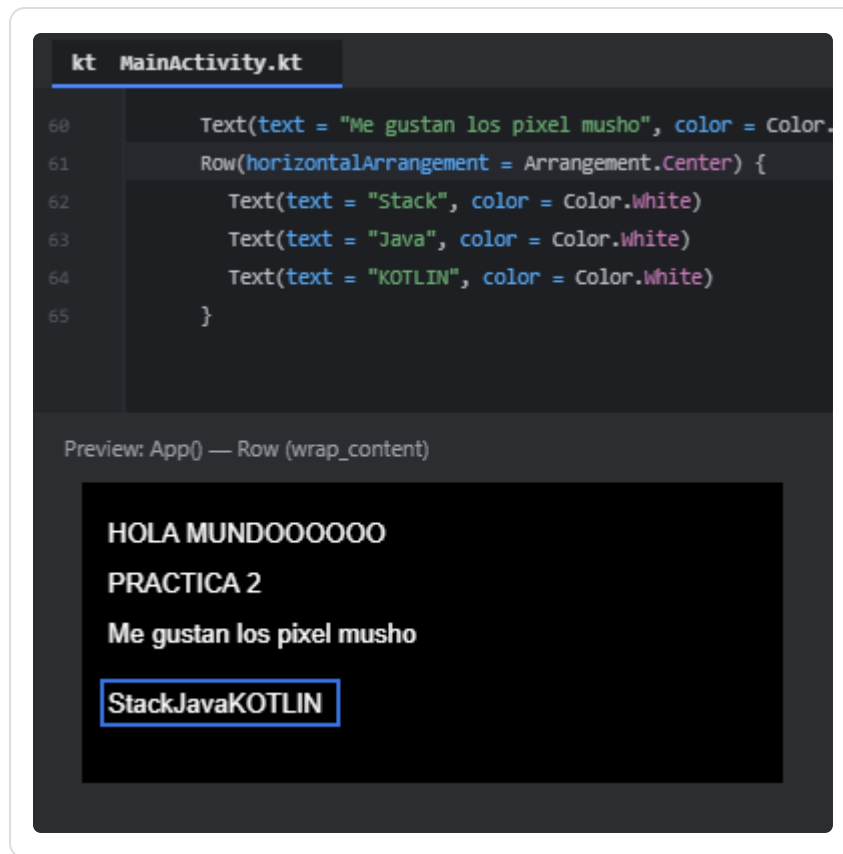


Ilustración 6: Comprobación del tamaño envolvente (wrap content) al usar `Arrangement.Center`

Para resolverlo, se añadió `modifier = Modifier.fillMaxWidth()` para que la fila abarcara todo el ancho de la pantalla (Ilustración 7) y posteriormente se cambió la disposición a `Arrangement.SpaceBetween`, logrando que el espacio horizontal total se repartiera proporcionalmente entre las tres palabras (Bloque de código 4 e Ilustración 8).

Bloque de código 4. Expansión de ancho y distribución con `Arrangement.SpaceBetween`

```

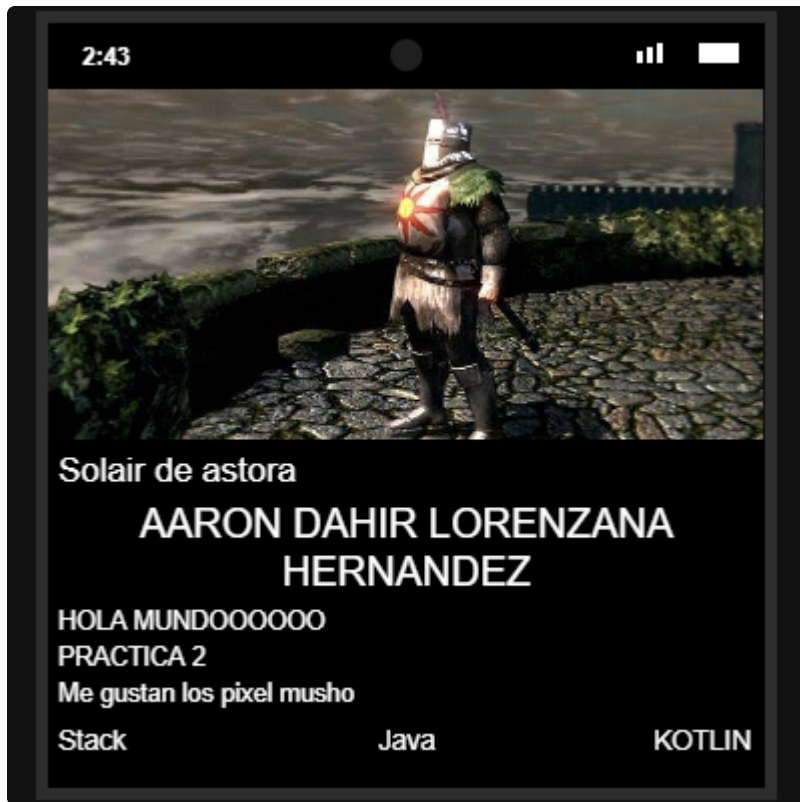
1 Row(
2     horizontalArrangement = Arrangement.SpaceBetween,
3     modifier = Modifier.fillMaxWidth()
4 ) {
5     Text(text = "Stack", color = Color.White)
6     Text(text = "Java", color = Color.White)
7     Text(text = "KOTLIN", color = Color.White)
8 }

```

Descripción: Al combinar `Modifier.fillMaxWidth()` con `Arrangement.SpaceBetween`, Compose distribuye los tres textos de forma equitativa a lo largo del ancho total de la vista.

```
kt MainActivity.kt

60     Text(text = "Me gustan los pixel musho", color = Color.White)
61
62     Row(
63         horizontalArrangement = Arrangement.Center,
64         modifier = Modifier.fillMaxWidth()
65     ) {
66         Text(text = "Stack", color = Color.White)
67         Text(text = "Java", color = Color.White)
68         Text(text = "KOTLIN", color = Color.White)
69     }
```

Ilustración 7: Expansión de la fila a todo el ancho con `Modifier.fillMaxWidth()`Ilustración 8: Distribución equitativa de elementos con `Arrangement.SpaceBetween`

3.4 Aplicación de márgenes internos (padding) y saturación de elementos

Para evitar que el primer y el último texto de la fila quedaran pegados a los bordes laterales del teléfono, se concatenó el modificador `.padding(16.dp)` al `Modifier` de la fila y se formateó el código (Bloque de código 5 e Ilustración 9).

Bloque de código 5. Adición de padding de 16 dp a la fila

```
1 Row(  
2     horizontalArrangement = Arrangement.SpaceBetween,  
3     modifier = Modifier  
4         .fillMaxWidth()  
5         .padding(16.dp)  
6 ) {  
7     Text(text = "Stack", color = Color.White)  
8     Text(text = "Java", color = Color.White)  
9     Text(text = "KOTLIN", color = Color.White)  
10 }
```

Descripción: En la línea 5 se encadena `.padding(16.dp)` para insertar un margen interno de 16 píxeles independientes de densidad alrededor de la fila.



Ilustración 9: Aplicación de `Modifier.padding(16.dp)` y formateo de código

A continuación, para evidenciar las limitaciones de un contenedor Row estático, se duplicaron múltiples componentes `Text(text = "SUSCRÍBETE", color = Color.White)` dentro de la fila (Ilustración 10). Al ejecutar la aplicación en el emulador, se observó que los textos excedieron el ancho físico de la pantalla, recortándose y amontonándose sin posibilidad de deslizar lateralmente (Ilustración 11).



Ilustración 10: Adición de múltiples etiquetas Text dentro del contenedor Row

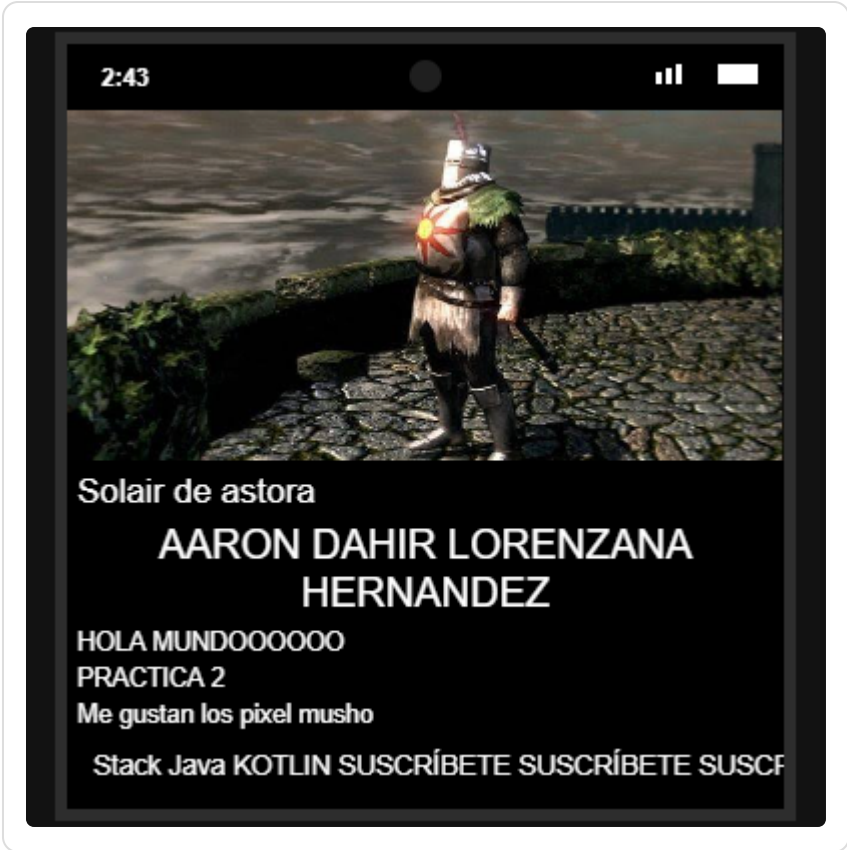


Ilustración 11: Problema de saturación y corte visual en la fila estática sin scroll

La Tabla 3 sintetiza los componentes, modificadores y propiedades utilizados en la configuración de las filas.

Tabla 3: Modificadores y propiedades aplicados a las filas de la práctica

Componente	Propiedad / Modificador	Función en la interfaz
Row / LazyRow	Modifier.fillMaxWidth()	Expande el contenedor horizontal para que abarque todo el ancho de la pantalla.
Row / LazyRow	.padding(16.dp)	Agrega un margen interno de 16 dp evitando que los textos toquen el borde de la pantalla.
Row	Arrangement.SpaceBetween	Distribuye los elementos de la fila dejando espacios iguales entre ellos.
LazyRow	Arrangement.spacedBy(16.dp)	Aplica una separación constante de 16 dp entre cada ítem desplazable de la fila.
LazyRow	Bloque item { ... }	Encapsula los elementos hijos dentro de la lista horizontal con reciclaje de memoria.

3.5 Implementación de desplazamiento horizontal con LazyRow

Para dotar a la fila de desplazamiento horizontal (*scroll*), se cambió el componente `Row` por `LazyRow`. Dado que `LazyRow` requiere estructurar su contenido mediante el DSL de listas perezosas, se colocaron los componentes `Text` dentro de bloques `item { ... }` en minúsculas (Bloque de código 6 e Ilustración 12).

Bloque de código 6. Transformación de `Row` a `LazyRow` con bloques `item`

```
1 LazyRow(  
2     horizontalArrangement = Arrangement.spacedBy(16.dp),  
3     modifier = Modifier  
4         .fillMaxWidth()  
5         .padding(16.dp)  
6 ) {  
7     item { Text(text = "Stack", color = Color.White) }  
8     item { Text(text = "Java", color = Color.White) }  
9     item { Text(text = "KOTLIN", color = Color.White) }  
10    item { Text(text = "SUSCRÍBETE", color = Color.White) }  
11    item { Text(text = "SUSCRÍBETE", color = Color.White) }  
12    item { Text(text = "SUSCRÍBETE", color = Color.White) }  
13    item { Text(text = "SUSCRÍBETE", color = Color.White) }  
14 }
```

Descripción: Al utilizar `LazyRow` y definir los textos en bloques `item` con `Arrangement.spacedBy(16.dp)`, la fila habilita el desplazamiento horizontal fluido sin desbordar la pantalla.

```

kt MainActivity.kt
69     LazyRow(
70         horizontalArrangement = Arrangement.spacedBy(16.dp),
71         modifier = Modifier
72             .fillMaxWidth()
73             .padding(16.dp)
74     ) {
75         item { Text(text = "Stack", color = Color.White) }
76         item { Text(text = "Java", color = Color.White) }
77         item { Text(text = "KOTLIN", color = Color.White) }
78         item { Text(text = "SUSCRÍBETE", color = Color.White) }
79         item { Text(text = "SUSCRÍBETE", color = Color.White) }
80     }

```

Ilustración 12: Sustitución de Row por LazyRow e incorporación del bloque item

3.6 Modularización en funciones Composable y prueba de scroll bidireccional

Finalmente, para mantener una arquitectura limpia y evitar el error de declarar funciones `@Composable` anidadas dentro de un bloque `item`, se extrajo el `LazyRow` hacia una función de nivel superior llamada `MyRowExample()` y se invocó al final del `LazyColumn` dentro de `App()` (Bloque de código 7 e Ilustración 13).

Bloque de código 7. Código completo final en MainActivity.kt con LazyColumn y LazyRow

```

1 package com.example.practica2
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.Image
7 import androidx.compose.foundation.background
8 import androidx.compose.foundation.layout.Arrangement
9 import androidx.compose.foundation.layout.fillMaxSize
10 import androidx.compose.foundation.layout.fillMaxWidth
11 import androidx.compose.foundation.layout.height
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.foundation.lazy.LazyColumn
14 import androidx.compose.foundation.lazy.LazyRow
15 import androidx.compose.material3.Text
16 import androidx.compose.runtime.Composable

```

```
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.graphics.Color
19 import androidx.compose.ui.res.painterResource
20 import androidx.compose.ui.text.style.TextAlign
21 import androidx.compose.ui.tooling.preview.Preview
22 import androidx.compose.ui.unit.dp
23 import androidx.compose.ui.unit.sp
24
25 class MainActivity : ComponentActivity() {
26     override fun onCreate(savedInstanceState: Bundle?) {
27         super.onCreate(savedInstanceState)
28         setContent {
29             App()
30         }
31     }
32 }
33
34 @Preview(showBackground = true)
35 @Composable
36 fun App() {
37     LazyColumn(
38         modifier = Modifier
39             .fillMaxSize()
40             .background(Color.Black)
41     ) {
42         item {
43             Image(
44                 painter = painterResource(id = R.drawable.images),
45                 contentDescription = "Solair de astora",
46                 modifier = Modifier
47                     .fillMaxWidth()
48                     .height(400.dp)
49             )
50             Text(text = "Solair de astora", fontSize = 24.sp, color = Color.White)
51             Text(
52                 text = "AARON DAHIR LORENZANA HERNANDEZ",
53                 fontSize = 32.sp,
54                 color = Color.White,
55                 modifier = Modifier.fillMaxWidth(),
56                 textAlign = TextAlign.Center
57             )
58             Text(text = "HOLA MUNDO000000", color = Color.White)
59             Text(text = "PRACTICA 2", color = Color.White)
60             Text(text = "Me gustan los pixel musho", color = Color.White)
61
62             MyRowExample()
63         }
64     }
65 }
66
67 @Composable
68 fun MyRowExample() {
```

```

69     LazyRow(
70         horizontalArrangement = Arrangement.spacedBy(16.dp),
71         modifier = Modifier
72             .fillMaxWidth()
73             .padding(16.dp)
74     ) {
75         item { Text(text = "Stack", color = Color.White) }
76         item { Text(text = "Java", color = Color.White) }
77         item { Text(text = "KOTLIN", color = Color.White) }
78         item { Text(text = "SUSCRÍBETE", color = Color.White) }
79         item { Text(text = "SUSCRÍBETE", color = Color.White) }
80         item { Text(text = "SUSCRÍBETE", color = Color.White) }
81         item { Text(text = "SUSCRÍBETE", color = Color.White) }
82         item { Text(text = "SUSCRÍBETE", color = Color.White) }
83         item { Text(text = "SUSCRÍBETE", color = Color.White) }
84         item { Text(text = "SUSCRÍBETE", color = Color.White) }
85     }
86 }

```

Descripción: En la línea 62 se invoca `MyRowExample()` dentro del ítem del `LazyColumn`, y en las líneas 67 a 86 se define la función composable independiente que renderiza el `LazyRow`.



Ilustración 13: Modularización del código mediante la función `MyRowExample()`

Al ejecutar la aplicación en el emulador, se comprobó que la fila inferior permite deslizar horizontalmente todas las etiquetas "SUSCRÍBETE" (Ilustración 14) y que, al girar el teléfono a modo horizontal, funcionan simultáneamente tanto el scroll vertical de la pantalla como el scroll horizontal de la fila (Ilustración 15).

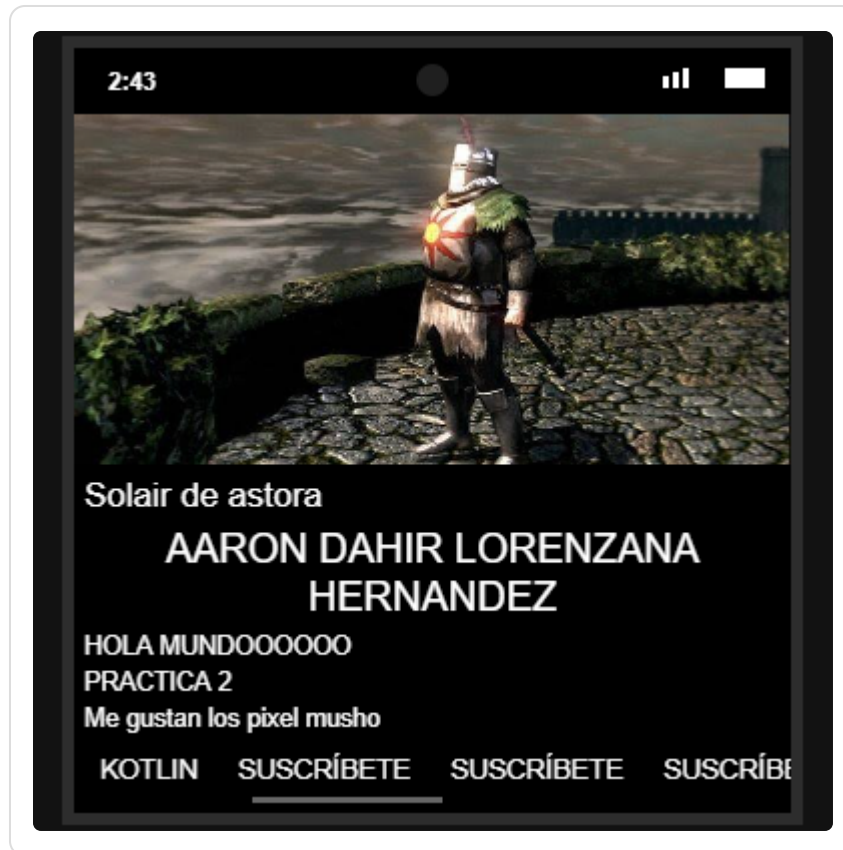


Ilustración 14: Prueba del desplazamiento horizontal (scroll) en el emulador

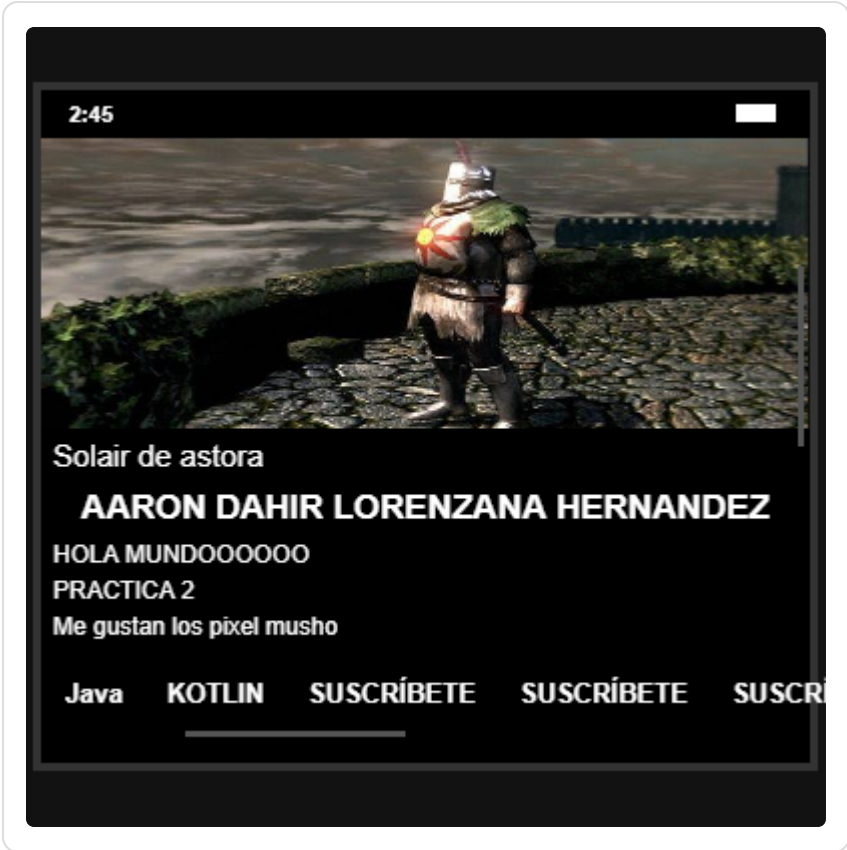


Ilustración 15: Funcionamiento simultáneo del scroll vertical y horizontal al rotar la pantalla

3.7 Análisis de resultados

Los resultados obtenidos validan experimentalmente el comportamiento de los contenedores horizontales en Jetpack Compose. Se verificó que un Row sin modificador de tamaño se ajusta únicamente al ancho de su contenido (Ilustraciones 5 y 6), por lo que requiere `Modifier.fillMaxWidth()` para que propiedades como `Arrangement.SpaceBetween` distribuyan los elementos en pantalla (Ilustraciones 7 y 8). Asimismo, se comprobó que ante listas extensas de elementos horizontales, sustituir Row por LazyRow habilita el desplazamiento lateral inmediato sin interferir con el desplazamiento vertical del LazyColumn padre (Ilustraciones 12, 14 y 15). La Tabla 4 vincula cada objetivo específico con sus evidencias.

Tabla 4: Relación entre objetivos específicos y evidencias

Objetivo específico	Apartado	Evidencias
1. Explicar Row y LazyRow frente a XML	2.1 y 2.2	Tabla 1
2. Integrar Row al final del LazyColumn previo	3.1 y 3.2	Ilustraciones 1, 2, 3, 4 y 5; Bloques 1 y 2
3. Configurar horizontalArrangement y fillMaxWidth()	3.3	Ilustraciones 6, 7 y 8; Bloques 3 y 4
4. Aplicar márgenes con Modifier.padding(16.dp)	3.4	Ilustración 9; Bloque 5; Tabla 3
5. Evidenciar la saturación en Row y migrar a LazyRow	3.4 y 3.5	Ilustraciones 10, 11 y 12; Bloque 6

Objetivo específico	Apartado	Evidencias
6. Modularizar en función Composable y probar scroll	3.6 y 3.7	Ilustraciones 13, 14 y 15; Bloque 7; Tabla 4

4. Conclusiones

4.1 Aportes

La práctica permitió cumplir satisfactoriamente el objetivo general al integrar maquetación horizontal con `Row` y `LazyRow` dentro del proyecto vertical previamente desarrollado. El principal aporte consistió en demostrar cómo combinar listas perezosas verticales y horizontales (scroll bidireccional), cómo distribuir elementos mediante `Arrangement` y cómo modularizar componentes en funciones `@Composable` independientes evitando errores de compilación.

4.2 Limitaciones

El alcance de esta práctica se enfocó en el diseño lineal en dos ejes (columnas y filas) con elementos declarados manualmente en bloques `item`, sin implementar aún la superposición de componentes en el eje Z (capas) ni el enlace con colecciones de datos dinámicas.

4.3 Recomendaciones y proyecciones

Como continuación del aprendizaje, se recomienda abordar el siguiente episodio del curso para estudiar el contenedor `Box` (cajas) y la combinación avanzada de filas, columnas y cajas para maquetar tarjetas y pantallas complejas en Android.

5. Referencias

Android Developers. (s. f.-a). *Aspectos básicos del diseño de Compose*. Google.
<https://developer.android.com/develop/ui/compose/layouts/basics?hl=es-419>

Android Developers. (s. f.-b). *Cómo cargar imágenes en Compose*. Google.
<https://developer.android.com/develop/ui/compose/graphics/images/loading?hl=es-419>

Android Developers. (s. f.-c). *Modificadores en Compose*. Google.
<https://developer.android.com/develop/ui/compose/modifiers?hl=es-419>

Android Developers. (s. f.-d). *Listas y cuadrículas en Compose*. Google.
<https://developer.android.com/develop/ui/compose/lists?hl=es-419>

Guimerá Orozco, A. [Programación by AristiDevs]. (2021, 30 de septiembre). *FILAS (ROW) en JETPACK COMPOSE - Curso Jetpack Compose desde cero - Tutorial Android COMPOSE [2021]* [Video]. YouTube.
<https://www.youtube.com/watch?v=tlhioKYvtGE>

JetBrains. (s. f.). *Kotlin docs*. <https://kotlinlang.org/docs/home.html>

6. Anexos

En este apartado se incluye material de apoyo que complementa el reporte técnico de la práctica.

Anexo A. Datos del video consultado

Título: FILAS (ROW) en JETPACK COMPOSE - Curso Jetpack Compose desde cero - Tutorial Android COMPOSE [2021]

Canal: Programación by AristiDevs (Mobile Dev by AristiDevs)

Enlace al video: <https://www.youtube.com/watch?v=tlhioKYvtGE>

Lista de reproducción: [2025] Curso Jetpack Compose desde cero, https://www.youtube.com/playlist?list=PL8ie04dqq7_ORKWlwiaSTcyBKtasZtNUD

Temas del video aplicados en la práctica: Maquetación horizontal con Row, alineación y distribución con horizontalArrangement (Arrangement.Center, Arrangement.SpaceBetween, Arrangement.spacedBy), uso de Modifier.fillMaxWidth() y Modifier.padding(16.dp), e implementación de scroll horizontal mediante LazyRow y bloques item.